MINISTERUL EDUCAȚIEI ȘI CERCETĂRII

UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA
FACULTATEA DE AUTOMATICĂ ŞI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

DESIGN AND IMPLEMENTATION OF A HEXAPOD ROBOT FOR EXPLORATION APPLICATIONS

LICENSE THESIS

Graduate: **Sima Răzvan-Gheorghe**

Scientific coordinator: **Conf. Dr. Ing. Roxana Rusu-Both**

2025

MINISTERUL EDUCAȚIEI ȘI CERCETĂRII

UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

DEAN,
Prof. dr. ing. Vlad MUREȘAN

DEPARTMENT DIRECTOR,
Prof. dr. ing. Honoriu VĂLEAN

Graduate: **Sima Răzvan-Gheorghe**

DESIGN AND IMPLEMENTATION OF A HEXAPOD ROBOT FOR EXPLORATION APPLICATIONS

1. **Project proposal:** *The focus of this project is to build a hexapod with limited budget that can still be used in limited exploration duties as well as education purposes.This project is focused on integrating both the hardware assembly and software control, including Inverse Kinematics, interpolation for smooth movement to be used in a tripod gait and a wireless command input to demonstrate a low-cost ,modular platform can achieve real-time control of the hexapod.*
2. **Project contents:** *Introduction , Legged robots , Analysis and design , Implementation and testing , Conclusion , Bibliography*
3. **Place of documentation**: Universitatea Tehnică din Cluj-Napoca,Departamentul Calculatoar
4. **Consultants**: Roxana Rusu-Both
5. **Data emiterii temei:** 11 Iulie 2025
6. **Data predării:** 11 Iulie 2025

Absolvent: _____

Coordonator științific: _____

MINISTERUL EDUCAȚIEI ȘI CERCETĂRII

UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA
FACULTATEA DE AUTOMATICĂ ŞI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

Declarație pe propria răspundere privind autenticitatea lucrării de licență

Subsemnatul **Sima Răzvan-Gheorghe**, legitimat(ă) cu CI seria XM nr. 045638 CNP 5020905245026, autorul lucrării DESIGN AND IMPLEMENTATION OF A HEXAPOD ROBOT FOR EXPLORATION APPLICATIONS elaborată în vederea susținerii examenului de finalizare a studiilor de licență la Facultatea de Automatică și Calculatoare, Specializarea Automatica si Infromatica Aplicata(in limba engleza) din cadrul Universității Tehnice din Cluj-Napoca, sesiunea iulie a anului universitar 2025, declar pe propria răspundere că această lucrare este rezultatul propriei activități intelectuale, pe baza cercetărilor mele și pe baza informațiilor obținute din surse care au fost citate, în textul lucrării, și în bibliografie.

Declar că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor.

Declar, de asemenea, că această lucrare nu a mai fost prezentată în fața unei alte comisii de examen de licență.

In cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administrative, respectiv, *anularea examenului de licență*.

Data	Nume, Prenume
11/07/2025	Sima Razvan Gheorghe

Semnătura

Sima

Cuprins

Chapter 1: Introduction **3**
1.1. Objectives **4**
1.1.1 Mechanical Design: 4
1.1.2 Hardware Architecture Design 4
1.1.3 Software Architecture Development: 5
1.1.4 Wireless Communication Interface 5
1.1.5 Power supply and Integration: 5
1.1.6 Testing and Validation: 5
1.1.7 Addresing Constraints and Limitations: 5
1.1.8 Inverse kinematic: 6
1.1.9 Motor Control 6
1.2. Specifications and Constraints: **7**
1.2.1. Specifications: 7
1.2.2. Constraints 7

Chapter 2 : Legged robots **9**
2.1. Examples of legged robots **9**
2.1.1. Hexapod Robot DYLEMA SIL06 [11] 9
2.1.2. Boston Dynamics – Spot [12] 11
2.1.3. Educational and Research Hexapods 11
2.1.4. 3D-Printed Hexapods 12
2.1.5. RHex – University of Michigan / Koditschek & Goldman [15] 14
2.1.6. Lauron V [16] 15
2.1.7. ANYmal – ETH Zürich / ANYbotics [17] 16
2.2. Robot kinematics **17**
2.2.1. Tripod Gait principle 17
2.3. Proposed solution **19**

Chapter 3 . Analysis and design **21**
3.1. Analysis of hardware components **21**
3.1.1. Walking motion through wireless joystick control 21
3.1.2. Component Tables 21
3.1.3. Joystick module: 22
3.1.4. Arduino nano: 22

3.1.5. NRF24L01 trans-receiver module: 23
3.1.6. PCA9685 Servo Drivers: 23
3.1.7. Arduino Mega: 23
3.1.8. MG996R actuators: 24
3.1.9. LM2596 Buck Converter: 25
3.1.10. Real-time movement interpolation and gait stability 25
3.2. Design of Hardware solution **26**
3.3. Analysis of software technologies and methods **27**
3.3.1. Motor Control: 27
3.3.2. Direct Kinematic: 28
3.3.3. Inverse kinematic: 28
3.3.4. Movement Control: 30
3.4. Design of software solutions **31**
3.4.1. System architecture 31
3.4.2. Low-level software architecture: 33
3.4.3. High Level software architecture: 33

Chapter 4 : Implementation and testing **35**
4.1. Hardware implementation of the prototype **35**
4.2. Implementation and testing of the software system **37**
4.2.1. Implementation and testing of motor control: 40
4.2.2. Implementation of the Direct kinematic 42
4.2.3. Implementation and Testing of Inverse Kinematics: 43
4.3. Hexapod system testing **44**

Chapter 5 Conclusion **46**
5.1. Achieved Results **46**
5.1.1. Degree of Achievement of Proposed Objectives : 46
5.1.2. List of personal contributions: 47
5.2. Future development direction **48**

Bibliography **49**

Chapter 1: Introduction

The endless endeavour for scientific innovation in the field of automation has led to significant progress in the use of robots across a wide range of industries from bakeries to some of the most advanced and precise domains, such as microcontroller manufacturing [1].

Among the many branches of robotics , mobile robots stand out for their ability to emulate life and operate in different environments.

While wheeled and tracked robots are popular in industrial logistics , legged robots are gaining the upper hand in traversing rougher terrain due to their ability to adapt better in terms of modularity and have greater stability. One popular configuration of a legged robot is a hexapod robot ,a six-legged structure inspired by the biological movements of insects.

Its symmetry, stability and ability to move even if one leg fails can make it an interesting platform for discussion for both research and real-world applications such as exploration,inspection and rescue missions in cases like cave explorers. By analyzing further the advantages of legged robots in some situations we can outline with almost pin-point accuracy that:

- In terms of stability and adaptability legged robots are more versatile while passing obstacles like gaps, rocks and roughish terrain.
- This is also because in comparison with other robots that have less legs or less modulable skeleton frame the hexapod weight is distributed onto 6 legs , where each legs has 3 joints which contribute to the movement of the robot and adjusts it's posture accordingly and balance itself according to the situation.
- While wheeled robots are often cheaper to produce their use usually requires investments in infrastructure such as flat or straight road surface , which greatly limit their flexibility.

Technological progress does not always require looking forward; at times nature itself can provide a solution to complex engineering problems.A notable example is the Japan train problem of air resistance when entering tunnels that lead to the creation of Shinkansen bullet train.

Engineers found inspiration for the creation of the bullet train in the kingfisher bird[2], whose beak allows it to dive into the water with minimal splash. By mimicking this natural geometry ,they successfully reduced tunnel sonic booms and improved energy efficiency.

Similarly, in robotics studying the locomotion of insects has led to advancements in legged robot design. The hexapod , modeled after insect movement, demonstrates how biological principles can be adapted for practical use in research in order for future engineers to learn of multiple joints robots and real-life exploration.

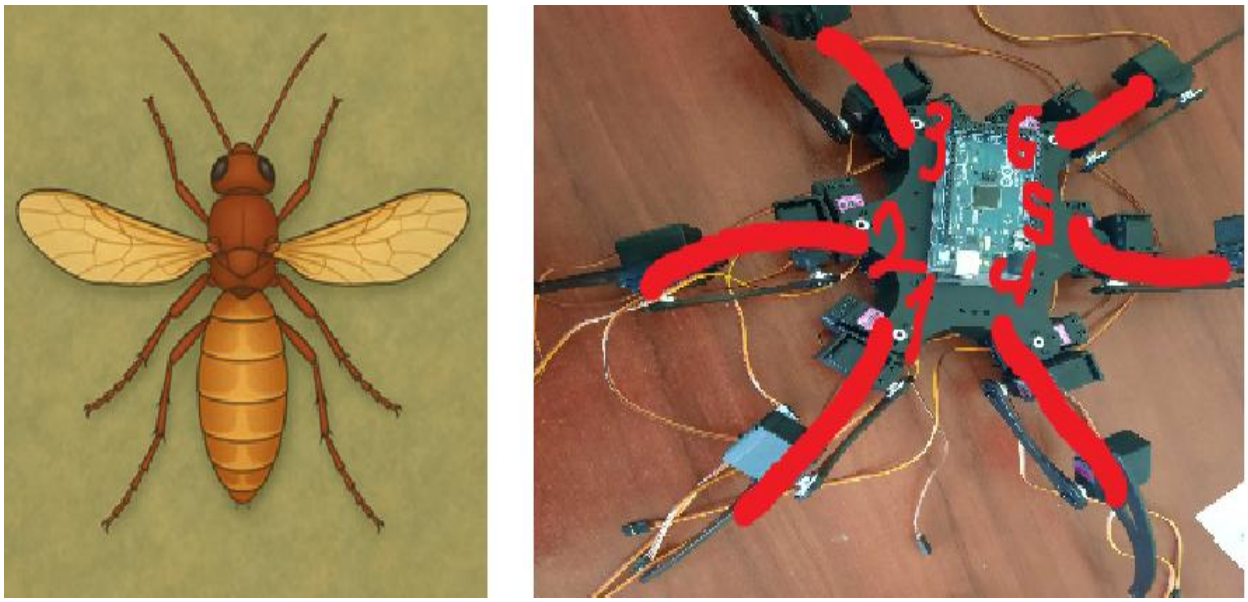

Figure 1. Design of hexapod compared to insect

In short we could say that this paper explores the design and development of a hexapod robot that mimics insectoid movements by integrating inverse kinematic and a custom-built wireless joystick controller which directs the robot movements which is further refined using interpolation technique to achieve smoother movement

1.1. Objectives

The main objective of this project is to implement a cost-effective hexapod robot capable of smooth, insect-like movement being capable of moving in a certain area without damaging it's components.

To achieve this general objective, the following specific objectives have been identified:

1.1.1 Mechanical Design:

- Design the hexapod's mechanical structure, including chassis, legs, and servo mounts.

- Choose materials that offer durability and low weight — for example, evaluating the use of aluminum over plastic to resist mechanical stress during walking.

1.1.2 Hardware Architecture Design:

- Select appropriate actuators (servo motors) with a balance of torque, price, and availability.

- Choose a suitable microcontroller (such as the Arduino Mega) capable of handling multi-channel servo control and interfacing with external modules.

- Integrate necessary communication modules for wireless control, such as the NRF24L01.

1.1.3 Software Architecture Development:

- Develop an inverse kinematic model for calculating joint angles of each leg based on a desired position in 3D space.

- Implement real-time control logic that translates joystick input into walking commands using a predefined gait (e.g., tripod gait).

- Use PWM signal drivers (e.g., PCA9685) to control all 18 servos precisely and simultaneously.

1.1.4 Wireless Communication Interface

- Design a wireless remote control system using a joystick and an Arduino Nano paired with an NRF24L01 module to send commands.
- Ensure robust communication between the transmitter and receiver using the SPI protocol and reliable data transmission mechanisms.

1.1.5 Power supply and Integration:

- Choose a reliable power source (e.g., 7.4V Li-Po battery) capable of supporting both the logic and actuation demands of the system.

- Design voltage regulation solutions (such as buck converters with filtering capacitors) to ensure stable operation of sensitive components like the NRF module.

1.1.6 Testing and Validation:

- Test individual modules (servo control, IK model, communication) independently before full system integration.

- Validate gait stability and motion coordination on various surfaces and under repeated movement conditions.

1.1.7 Addresing Constraints and Limitations:

- Explore affordable components due to budget constraints while still aiming for functional and educational relevance.

- Account for known limitations of budget servos like MG996R and plan design compensations (mechanical repositioning, smoother gait transitions).

1.1.8 Inverse kinematic:

In the context of multi-legged robots such as hexapods, various motion control strategies can be considered for moving the legs. While several approaches exist — including predefined trajectories or direct control of joint angles inverse kinematics is one of the most commonly used methods, particularly for robots with more than two legs.

This is due to its ability to provide precise foot placement and coordinated movement, which are crucial for maintaining stability during locomotion.

At this stage, the goal is to explore and evaluate different kinematic modeling approaches, such as:

- Mathematical modeling (including direct and inverse kinematics)
- Motor control strategies
- Structural design and layout of the robot

These aspects will help shape the final implementation, which will be decided after further research and testing.

1.1.9 Motor Control

An essential component of the hexapod robot's hardware architecture is the motor control system**,** which must be capable of handling a total of 18 servos (3 per leg × 6 legs).
Several motor driver options are being considered for this purpose. One solution involves using a 32-channel servo driver, which simplifies wiring and control but comes at a relatively high cost.

An alternative and more cost-effective approach is based on the PCA9685**,** a 16-channel PWM driver. By using two PCA9685 boards in series, it's possible to effectively simulate a 32-channel system, making it suitable for controlling all 18 servos required by the hexapod.

This approach significantly reduces costs, as a PCA9685 module is priced at approximately 25 RON, offering a highly affordable and scalable solution for servo control. Further testing and integration will help determine whether the dual-PCA9685 setup meets timing, power, and performance requirements for stable movement.

In designing the mechanical structure of the hexapod robot, one of the key decisions involves the choice of materials for the chassis and leg components.
A 3D-printed plastic frame offers the advantage of being lightweight and easy to prototype. However, various sources and user experiences indicate that the joints of

3D-printed parts tend to wear out or break more easily, especially under the repeated stress typical of walking robots.

In contrast, a frame built from aluminum provides significantly better durability and rigidity, particularly around the servo joints, where structural integrity is crucial. The trade-off, however, is that aluminum is slightly heavier, requiring more powerful servos to maintain performance and lift the robot's body effectively.

Considering the need for long-term reliability and the importance of structural stiffness in a walking robot, the aluminum frame appears to be the more suitable option, even if it demands stronger actuators.

1.2. Specifications and Constraints:

When developing a hexapod robot intended for hands-on educational use, it is helpful to define a hypothetical yet realistic context that guides the design process. Consider a scenario in which a technical high school or a university engineering department seeks to introduce students to key robotics principles such as legged locomotion, kinematic modeling, and servo control through a modular and affordable hexapod platform.

In such a case, the robot must meet a set of functional requirements and operate within several practical constraints, such as:

1.2.1. Specifications:

- The robot should support 3 degrees of freedom per leg, enabling exploration of inverse kinematics.
- It must be capable of smooth and stable walking, ideally using interpolation or ramped motion control.
- The robot should be reprogrammable and modular, allowing students to modify or replace components.
- It should be safe for classroom use, with low-voltage components and minimal pinch points.

1.2.2. Constraints

- The robot must be built within a maximum of 1800 RON per unit — a limit aligned with what an individual student or department could realistically allocate.
- Parts should be easy to replace or repair using accessible components.
- The robot should withstand repeated handling and usage in a learning environment.
- It must be compact and portable, yet mechanically stable.

With these constraints in mind, design choices were made that prioritize cost-effectiveness without sacrificing educational value.
For example, while high-end 32-channel servo drivers exist, their cost would exceed the available budget. Instead, a more affordable solution was chosen: two PCA9685 16-

channel PWM drivers used in series offering the necessary 18 channels at a significantly lower price while still meeting performance needs.

Similarly, although 3D-printed plastic frames are lightweight and easy to prototype, they often suffer from mechanical wear, especially at joint connections. Therefore, a more robust aluminum frame was selected despite its higher weight, due to its superior durability and mechanical stiffness even if this trade-off required slightly more powerful servos to compensate.

By designing within these constraints and aligning with these functional requirements, the robot remains an accessible, safe, and effective educational platform while providing the flexibility to scale up in better-funded environments.

Chapter 2 : Legged robots

A hexapod robot is a mechanical and electronic device whose movement relies on six articulated legs. Compared to other legged robots such as bipeds, tripeds, or quadrupeds the hexapod offers superior flexibility and stability.

Its behavior is more complex, as not all six legs are needed for locomotion at once. The remaining legs can be used for tasks such as object manipulation or to improve maneuverability toward specific targets.

This type of robot was initially developed to test biological theories related to insect locomotion and motor control. Hexapod robots are particularly suitable for exploration or research missions in areas that are difficult or dangerous for humans to access, such as regions affected by earthquakes or other natural disasters.

In terms of leg arrangement, hexapod designs can vary, but most commonly, the legs are symmetrically distributed around the central body. Their movement is governed by different gait patterns, such as:

- **Alternating tripod gait** (using three legs per step),
- **Tetrapod or creeping gait** (moving one leg at a time).

2.1. Examples of legged robots

In recent decades, legged robots have become a growing field of research, with diverse applications in exploration, hazardous environment intervention, agriculture, and logistics. Among them, hexapod robots those with six legs—form an important subcategory due to the increased stability provided by their mechanical configuration.

2.1.1. Hexapod Robot DYLEMA SIL06 [11]

A notable example of practical hexapod implementation is the DYLEMA SIL06[11] robot, developed as part of the European DYLEMA project ("Dynamically Learning Multibody Agents") funded by the Horizon 2020 program. The robot was designed as an experimental platform for testing advanced adaptive control and machine learning algorithms applied to multibody systems, especially those with complex mobility such as legged robots.

Design and Architecture

SIL06 features a classical hexapod layout with six articulated limbs, each offering three degrees of freedom—horizontal rotation, vertical lift/drop, and extension/retraction. Its structure is built from lightweight materials such as aluminum and polymer composites, ensuring a good balance between rigidity and mass.

Each joint is actuated by intelligent servomotors equipped with position sensors, allowing for precise control. The central body hosts an onboard processing unit (e.g., NVIDIA Jetson), alongside an array of inertial sensors (IMU), pressure sensors for contact force measurement, and optionally, visual perception cameras.

Purpose and Innovation

The core aim of the DYLEMA[11] project is to develop online machine learning techniques that enable the robot to adapt its locomotion in real time based on environmental changes or mechanical failures. SIL06 serves as a testbed for these concepts, focusing on:

- Learning locomotion patterns through sensory feedback.
- Adapting posture and body configuration for dynamic stabilization.
- Achieving emergent behavior without explicitly programming each movement.

One of SIL06's[11] distinctive features is its combined use of Policy Gradient Reinforcement Learning and Model Predictive Control, enabling efficient and robust motion planning.

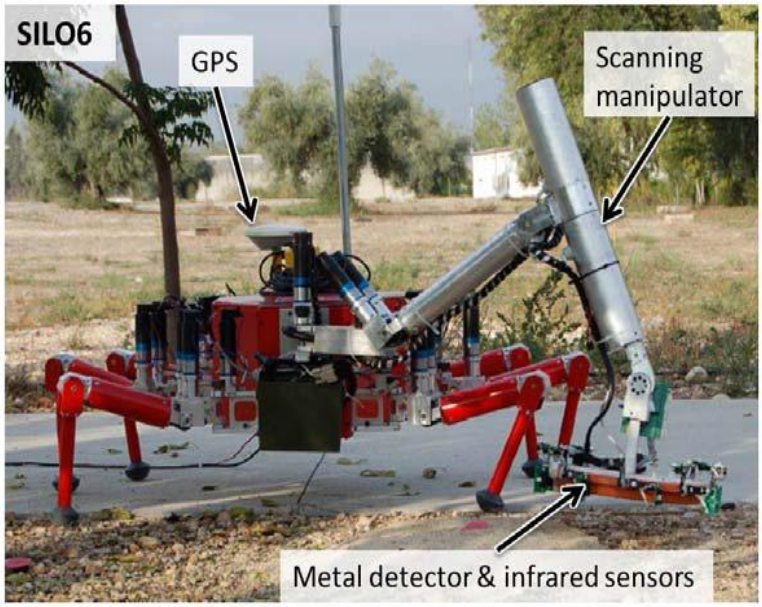

Figure 2. SIL06 Design [11]

Advantages:

- Supports real-world testing of modern learning algorithms, not just in simulation.
- Modular mechanical design, adaptable to varied test scenarios.
- Equipped with advanced sensors offering detailed interaction and motion data.

Limitations:

- Requires significant computational infrastructure to run real-time algorithms.
- Mechanical wear is a concern due to repeated testing cycles.
- Complex design limits reproducibility outside specialized labs.

The robot needs to be sufficiently large to integrate all its subsystems, including the onboard computer, electronics boards, motor drivers, DC motors, GPS module, batteries, manipulator arm, and other components.

The figure of SIL06 illustrates that the robot equipped with a manipulator arm, infrared sensors, GPS, and a metal detector.

2.1.2. Boston Dynamics – Spot [12]

Although Spot is a quadruped and not a hexapod, it is relevant as a benchmark in legged robot development, setting high standards for mobility, robustness, and adaptability. Its construction uses lightweight materials, precision electric actuators, and a robust frame, enabling it to operate in challenging environments.

Figure 3. Boston dynamic robot dog [12]

Advantages : High stability on uneven terrain

- relatively long energy autonomy
- integrated obstacle avoidance sensors.

Disadvantages :

- High production and maintenance costs
- complex manufacturing

2.1.3. Educational and Research Hexapods

A representative model is the PhantomX AX Metal Hexapod Mark II [13] from Trossen Robotics[13]. It uses Dynamixel servomotors and an aluminum frame, commonly used in education and research.

Advantages :

- Modular, customizable

- Moderately priced.

Disadvantages :

- Limited performance in heavy-duty tasks
- less stable at high speeds.

Fig 4. PhantomX AX Metal Hexapod Mark II [13]

2.1.4. 3D-Printed Hexapods

Many DIY and academic projects leverage 3D printing to reduce costs and increase design flexibility. This allows for rapid testing of various mechanical architectures.

Advantages :

- Low cost
- fast customization.

Disadvantages :

- Limited mechanical strength
- reduced durability in real-world applications.

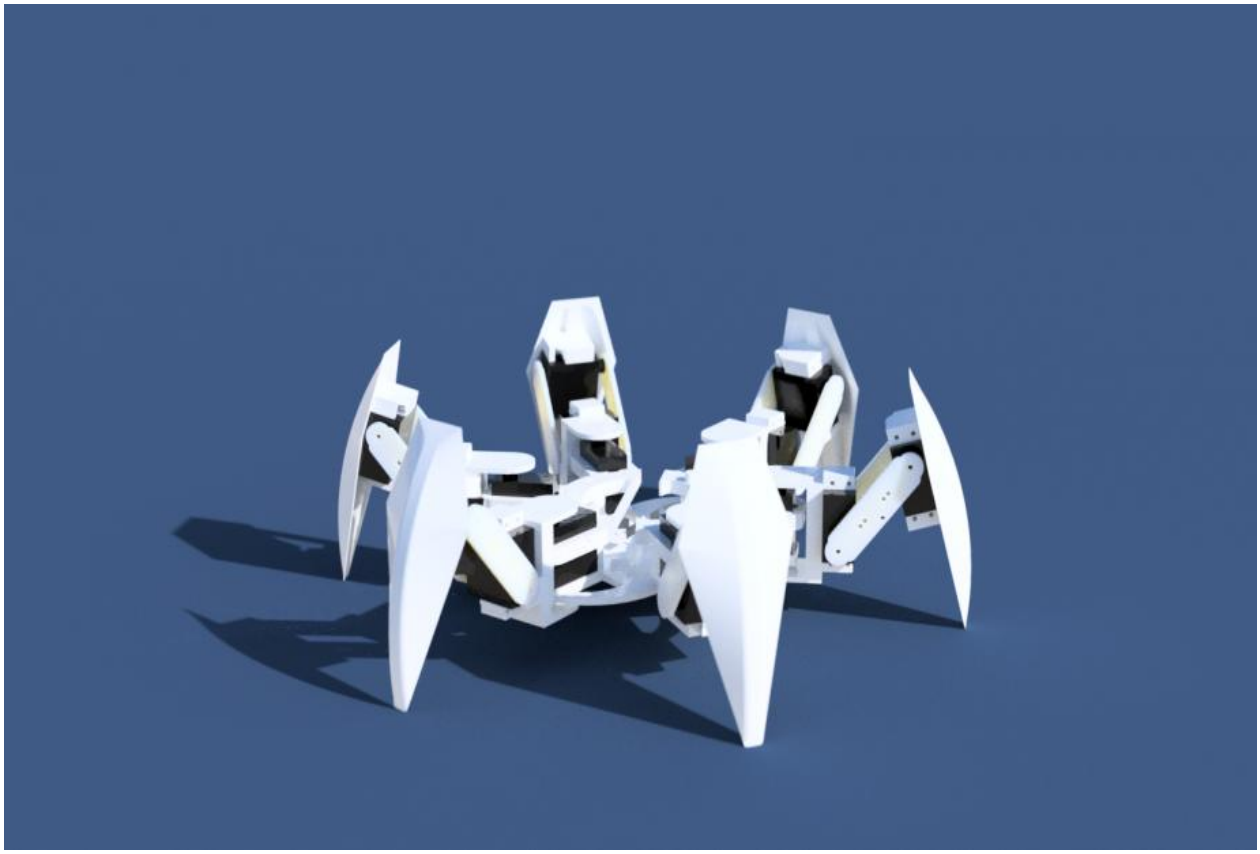

Fig 5. Hexapod printed concept [14]

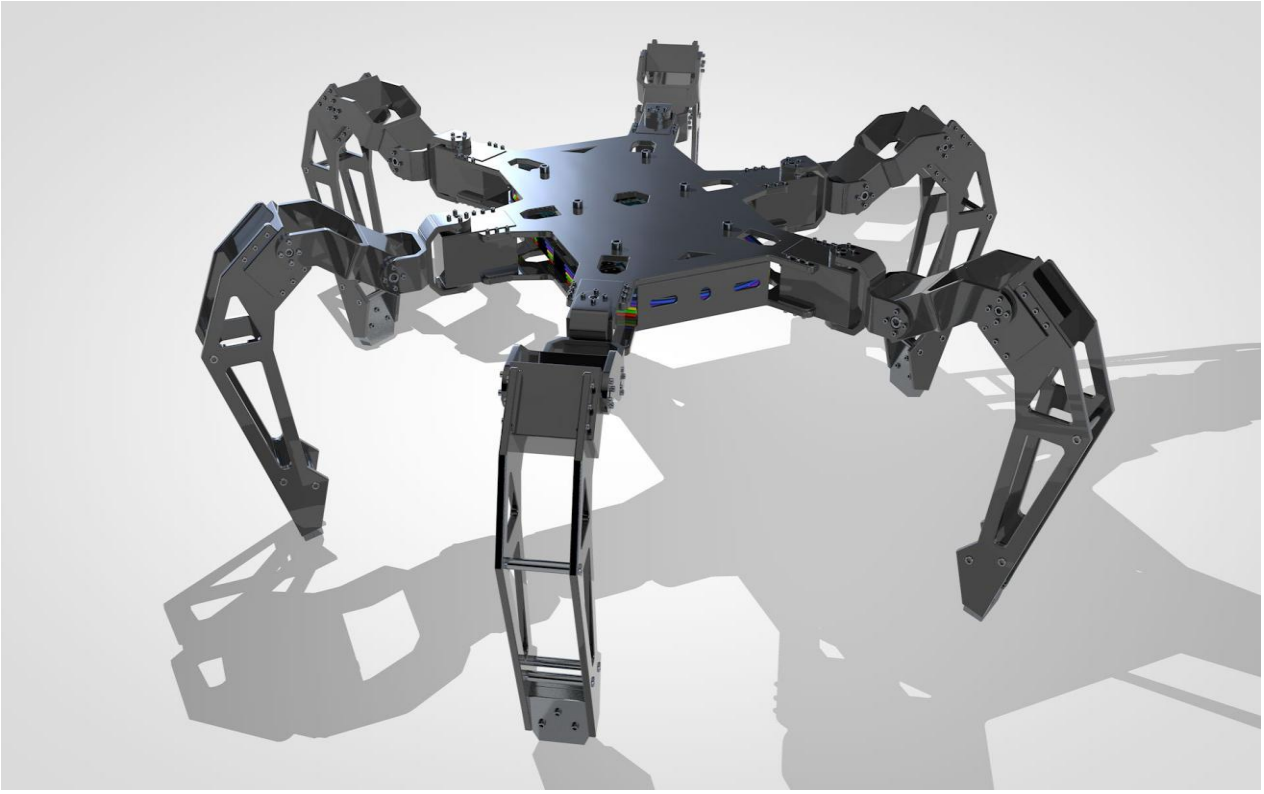

Fig 6. Hexapod Concept 3D [20]

2.1.5. RHex – University of Michigan / Koditschek & Goldman [15]

RHex is a hexapod with semi-rigid legs, designed for robust movement on rough terrain and inspired by insect locomotion.[15]

Contributions: Tested various gaits (bound, trot, gallop), demonstrated adaptive behavior in real environments.

Advantages:

- Capable of navigating uneven, rocky, muddy, and steep terrain, including stairs and rubble, thanks to its compliant legs and robust locomotion.
- Uses rotary legs without multiple joints, reducing complexity, weight, and mechanical failure points.
- One of the fastest hexapod robots in its class. Its dynamic gaits enable efficient forward motion, jumping, and even swimming in water.
- Extensively tested outdoors, making it suitable for tasks like search and rescue**,** military reconnaissance, and environmental monitoring**.**

Disadvantages:

- The rotary leg design lacks fine control compared to robots with individually actuated joints, making precision tasks or controlled manipulation difficult.
- Uses open-loop or precomputed dynamic gaits, which may not be as adaptive or responsive to unpredicted changes as gait systems using inverse kinematics.
- Despite simpler mechanics, the control software (especially for stability and terrain adaptation) can be complex and computationally demanding.

Fig 7. From RHEX a simple a Highly Mobile Hexapod Robot [15]

2.1.6. Lauron V [16]

The Lauron (Legged Anthropomorphic Utility Robot) project has reached its fifth version and is used for industrial inspection and intervention tasks.

- Contributions: Combines preprogrammed walking with sensor-based adaptation (IMU, force sensors).

Advantages:

- Adaptive Terrain Navigation:
  Designed for semi-autonomous exploration in challenging, uneven, and unstructured environments such as forests, rubble, or disaster zones.
- Biologically Inspired Design:
  Mimics insect-like walking and posture control, allowing efficient and stable locomotion on various surfaces.
- Force/Torque Feedback:
  Equipped with force sensors on its legs, enabling compliant walking and dynamic obstacle response without needing perfect terrain mapping.
- Advanced Gait Algorithms:
  Capable of multiple walking patterns including tripod, wave, and creep gaits, allowing it to adapt to surface irregularities and payload demands.
- Modular Architecture:
  Modular mechanical and software systems make it easier to test different sensors, gaits, and control strategies.

Disadvantages:

- Research-Oriented:
  LAURON V is not a commercial product and requires extensive robotics knowledge to operate, including real-time control systems and kinematics.
- Not Cost-Efficient:
  Built with precision sensors and actuators for research accuracy, making it expensive and unsuitable for budget-constrained applications like education.
- Limited Availability:
  Not mass-produced or open-source; mostly used within academic collaborations or research institutions.

Fig 8.Six-legged walking robot LAURON IV[16]

2.1.7. ANYmal – ETH Zürich / ANYbotics [17]

ANYmal is a quadruped robot developed by ETH Zürich, used for autonomous inspection in industrial facilities. It combines vision, lidar, and AI for path planning.[17]

- Contributions**:** Real-time active balancing and force control on each leg.

Advantages:

- Equipped with LiDAR, cameras, and IMUs, ANYmal can autonomously map and traverse unknown or complex environments without human intervention.
- Thanks to its torque-controlled joints and robust mechanical design, ANYmal can walk, trot, climb stairs, and even recover from falls.
- Comes with a range of sensors (LiDAR, thermal, RGB cameras, GPS), allowing it to perform detailed inspections in industrial scenarios.
- Offers 2–4 hours of operation depending on the terrain and mission profile.
- IP67-rated design allows operation in rain, mud, and dust, making it ideal for outdoor or industrial inspection tasks.

Disadvantages:

- As a research and commercial-grade robot, ANYmal is extremely expensive, making it inaccessible for educational or hobbyist applications.
- Requires high-level software knowledge and powerful onboard computation, not suitable for beginner-level robotics.
- Larger and heavier than typical educational robots; harder to transport and deploy in small-scale lab environments.

Figure 9. ANYmal[17]

2.2. Robot kinematics

„Kinematics is the study of motion without regarding to the forces that create it”[6]. Forward kinematic is the mathematical representation of the end effector based on the geometry and joint angle orientation. For forward kinematic , you can sum it up to the ideea that if you give known values to each joint you can find out where would the end effector end up.

While forward kinematic is useful for simulations and to check joint movements , it is less practical in real-life robotic applications where the desired position of the end-effector is known , but the required joint angles are not. In thse cases , inverse kinematic is the answer.

„Inverse kinematic is the inverse of a forward kinematic model”[3] , that is used to calculate the joint values for a specific position of the end-effector in 3D space. This method is widely used in legged robots , robotic arms and other manipulators where precise movements and coordination is required.

In the case of a hexapod , inverse kinematic enables each leg to move it’s foot along a defined path by computing the exact angles for coxa,femur and tibia joints. The implementation and calculation of the inverse kinematic for 3 join leg will be shown in more detail in the next chapter.

With the inverse kinematics logic established, we can now proceed to the gait implementation, which coordinates the leg movements to generate stable and repeatable walking motion.

2.2.1. Tripod Gait principle

In this project, a tripod gait was implemented since it is known to be among the most stable form of movement for a six-legged robots. In this gait, the legs are divided

into 2 groups so that only 3 legs move and 3 legs maintain stability, these groups alternating between each other when moving so that the robot doesn't fall down.

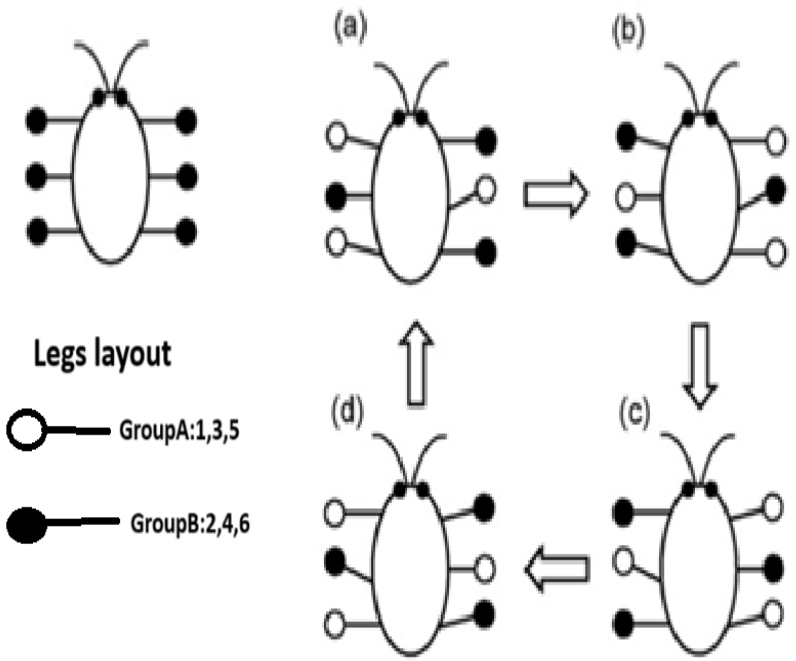

Figure 10. Tripod Gait movement

Based on the input recieved from the joystick that dictates when the robot moves , one group enters a swing phase whereas the other maintains position so that it can maintain the stability of the robot.Afterwards, after a group has moved to the final position the other group lifts itself in the air so that the other group that just finished its stepPath can push itself without the resistance of the leg group that would otherwise drag it's legs during this.

This alternating pattern allows for continous , fluid and stable forward motion while also minimizing the risk of tipping due to puting too much mechanical pressure on a single leg.

The tripod gait is further refined by using interpolation tehnique using the Ramp library[7] that makes the robot not reach the inputed value instantly , instead they gradually reach the proposed position , making walking more smoothly and therefore not affecting stability when moving.

To compare this with a gait movement without interpolation it would look like the robot would have jerky movements , lesser stability and increased wear on the servos as it makes the servos move to a direction very fast and in case of obstacles it can seriously hurt itself during testing period.

Therefore the use of the interpolation is a vital part of the hexapod movement for efficiency of robot walking movement and durability of the servos.

2.3. Proposed solution

Based on the review of the existing hexapods systems and control strategies presented in A, we can conclude that in terms of mechanical structures and resilience , this project proposes a cost effective , modular and education worthy hexapod robot.

The main goals is to provide a robot system that demonstrates key robotic concepts like inverse kinematic ,gait control and interpolation while at the same time providing an accessible cost and regional componental availability.

The robot design uses MG996R actuators that offers a balanced approach in term of price , torque and local availability. Even though these servos don't provide a higher torque than other high-end models, they provide sufficient torque and reliability for the robot to move efficiently enough.

The servos regional availability also ensures that replacements are more affordable and faster to arrive, making it perfect for constant use as it is happening in an educational institution.

The control of servos is achieved through the arduino Mega board , which coordinates the movement of all 18 servos through the PCA9685 PWM drivers, connected over the I^2C bus.

Furthermore, a custom-build joystick that uses a 9V battery and is equipped with a arduino nano that supplies 3.3V to the NRF24L01 trans-reciever module and reads the data sent from the analog pins of the joystick and sends it to the transmiter module to pass it onto the reciever module through radio, so that it can be read by the arduino Mega and use the data to execute a command defined within the program.

The arduino mega reciever NRF module is further safeguarded by a buck converter that has an input of 5V from the arduino Mega and is set on a constant 3.3V so that the module NRF has a constant supply of voltage which is even further guarded by a 47µF capacitor in case of voltage drop and it also has another use of filtering out noise as there has been a problem in the process of developing the reciever due to the power supply having voltage drops.

To enhance the movement of the robot and reduce mechanical stress on the gearbox of the actuators , the robot uses Ramp library [7] to smoothen the transition between joint angles. Rather than jumping from angle to another , the actuators movements are interpolated over time creating a fluid motion during walking.

For locomotion, the robot implements a tripod gait, which is widely used for six legged robots that use a stable walking pattern, ensuring stability.

The gait ensure that three legs remain stationary to stabilise the robot and 3 legs move forward untill they reach their final step of their stepPath , afterward the other group lifts it's legs and the first group that reached the end of the stepPath pushes the robot forward , emulating an tarantula like movement.

Mechanically , the robot is build with an aluminium frame that was order from China, chosen for it’s resilience compared to plastic ones ,especially under repeated mechanical stress. The design includes enough space to house the battery , electronics and servo wirings while maintaining a stable movement.

The control program running on arduino Mega is structured to listen constantly to the radio and if the reciever module recieves data within a certain range ,because since the joystick is basically 2 potentiometers put together the data sent would be up till 1080 , so therefore you can mimic with your joystick if for example you read X value > 100 and Y < 100 , on the real life joystick it would mean to push it forward, in program it would mean calling the moving forward command.

The data sent by the joystick would be analog , which would be read by the arduino nano that is programmed to tell the NRF module to act as a transmitter and not as a reciever in order to transmit the data to the arduino Mega.

Overall , the proposed solution of the robot prioritises affordability ,functionality and simplicity in its structure , which offers a good educational approach for the students in laboratories to learn more about robotics and inverse kinematics used in mobile robots that uses 3 joints.

It also servers as a good base for other students that intend to do research at universities and have more time on their hands to implement more upgrade such as a gyroscope with PID in order for the robot to stabilise better on uneven surface[5].

Chapter 3 . Analysis and design

3.1. Analysis of hardware components

3.1.1. Walking motion through wireless joystick control

The primary functionality of the hexapod robot is to perform insect-like walking movement based on real-time input received from a wireless joystick. The robot receives X and Y axis values from a joystick connected to an Arduino Nano, which communicates via NRF24L01 transceiver modules using SPI protocol.

These values determine whether the robot moves forward, stops, or eventually turns, depending on future extensions.

To achieve this, several hardware components are essential:

- Arduino Mega 2560: Used as the robot's main controller.
- NRF24L01 (Tx/Rx): Handles wireless transmission between joystick and robot.
- Joystick Module: Captures directional user input.
- Servo motors (MG996R): Power each joint of the six legs.
- PCA9685 (x2): Used to control up to 32 PWM channels for the 18 servos.

Once data is received, the microcontroller processes it, triggers inverse kinematics calculations for each leg, and sends PWM signals through the PCA9685 boards to control the legs accordingly. This allows the robot to walk using a tripod gait (3 legs lift while 3 remain grounded).

3.1.2. Component Tables

Nr.	Component	Used for
1	Joystick module	Capturing directional input from the user
2	Arduino Nano	Processing joystick input and sending RF signal
3	NRF24L01 (Transmitter)	Transmitting joystick data wirelessly
4	NRF24L01 (Receiver)	Receiving commands on the robot
5	Arduino Mega 2560	Processing movement logic and inverse kinematics
6	PCA9685 Driver (0x40)	Controlling servos on Group B legs
7	PCA9685 Driver (0x41)	Controlling servos on Group A legs
8	MG996R Servo Motors	Moving each leg joint (coxa, femur, tibia)
9	Power Supply (7.4V,5000mAh Li-po + Buck)	Supplying power to servos and logic units

3.1.3. Joystick module:

The joystick module is a key input device in this project, enabling the user to intuitively control the movement of the hexapod robot. It typically consists of two potentiometers (one for the X-axis and one for the Y-axis) and a push button integrated into the board. The X and Y axes represent horizontal and vertical directions, and their analog values range from 0 to 1023, depending on the position of the stick.

When the joystick is moved along the X or Y axis, it produces varying voltage levels that are read by the Arduino Nano through its analog input pins.

Once the analog readings are acquired, the Arduino Nano packages the data and sends it via the NRF24L01 transmitter module. This wireless transmission ensures that the user can operate the robot remotely, untethered by cables.

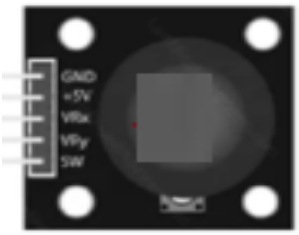

Fig 11. Joystick module

3.1.4. Arduino nano:

The Arduino Nano serves as the central processing unit on the transmitter side of the hexapod robot control system. It is a compact, breadboard-friendly microcontroller board based on the ATmega328P, offering the same core functionality as an Arduino Uno but in a much smaller form factor. This makes it ideal for lightweight, portable, or space-constrained applications—such as a handheld joystick controller.

In this project, the Arduino Nano performs several crucial roles:

- Data Acquisition: It continuously reads analog signals from the joystick's X and Y axes, as well as the digital signal from the push button. These readings reflect the user's desired movement direction and actions.
- Data Processing: The Nano interprets the analog joystick values and organizes them into a custom-defined data structure (like a struct JoystickData in code) to ensure the information is properly formatted for transmission.
- Wireless Communication: The Arduino Nano communicates with the NRF24L01 transmitter module using the SPI (Serial Peripheral Interface) protocol. It sends the joystick data wirelessly to the robot's receiver unit (another NRF24L01 connected to the Arduino Mega).

- Power Distribution: The Nano also acts as the power provider for the joystick module and NRF24L01. In this setup, it is typically powered by a 9V battery through the VIN pin, and it regulates voltage to 5V and 3.3V internally for its peripherals.

By managing these tasks, the Arduino Nano enables real-time, wireless control of the robot, playing a critical role in the user interface layer of the system.

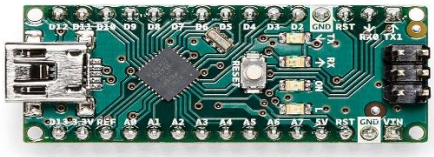

Fig 12 .Arduino nano

3.1.5. NRF24L01 trans-receiver module:

The NRF24L01 is a low-cost, highly efficient 2.4 GHz transceiver module used for wireless communication between microcontrollers. In this project, a pair of NRF24L01 modules enables wireless control of the hexapod robot from a remote joystick interface.

- On the transmitter side, the module is connected to an Arduino Nano, which sends joystick position data over SPI.
- On the receiver side, it connects to the Arduino Mega and listens for incoming commands, allowing the robot to respond in real time.

Its high data rate and low power consumption make it ideal for applications requiring fast, responsive communication like robotics. The modules must be tuned to the same **channel and address** to communicate reliably.

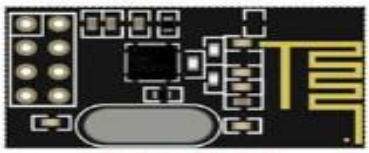

Fig 13.NRF24L01 trans-reciever module

3.1.6. PCA9685 Servo Drivers:

The PCA9685 is a 16-channel PWM driver board controlled via I²C communication. In this project, two PCA9685 boards are used (addresses 0x40 and 0x41) to drive 18 MG996R servo motors—3 for each of the 6 hexapod legs.

Key roles:

- Converts calculated joint angles (from inverse kinematics) into PWM signals.
- Sends those signals to each servo to position the joints precisely.
- Offloads timing-sensitive PWM generation from the main microcontroller, allowing smoother motion and reduced CPU load on the Arduino Mega.
- By using two synchronized boards, the system can control all legs independently and precisely, making coordinated insect-like motion possible.

3.1.7. Arduino Mega:

The Arduino Mega 2560 serves as the central microcontroller of the hexapod robot. With its 54 digital I/O pins, 16 analog inputs, and increased memory and processing power, it is ideal for handling complex tasks such as:

- Receiving joystick commands via the NRF24L01 receiver using SPI.

- Calculating inverse kinematics to determine joint angles for each leg.
- Controlling the interpolation logic through the Ramp library for smooth leg movement.
- Communicating with two PCA9685 drivers over the I²C protocol to control up to 18 servos simultaneously.

Its versatility, combined with ample hardware resources, makes it well suited for multitasking and real-time robotic motion coordination.

Fig 14.Arduino Mega

3.1.8. MG996R actuators:

The MG996R[8] is a high-torque standard servo motor widely used in robotics applications. Each leg of the hexapod uses three MG996R servos—one for each joint (coxa, femur, tibia)—for a total of 18 across the robot.

Key characteristics[8]:

- Torque: ~10–11 kg·cm, suitable for moderate loads.
- Operating voltage: 4.8V–6.0V (powered via external battery).
- Controlled via PWM signals from the PCA9685 drivers.

Although more affordable than premium servos, MG996Rs provide sufficient torque for basic walking functions. Their performance was balanced against cost, making them a practical choice for a student-designed prototype.

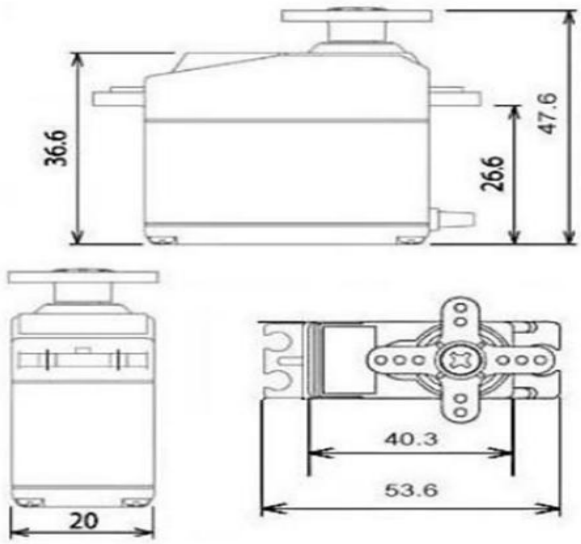

Fig 15.MG996R schematic

3.1.9. LM2596 Buck Converter:

The LM2596 is a step-down (buck) voltage regulator used to convert 5V to 3.3V. It is essential for powering the NRF24L01 module, which operates best at a stable 3.3V.

In this project:

- It receives 5V from the Arduino Mega as input.
- It outputs a regulated 3.3V, further filtered by a 47µF capacitor to suppress voltage drops and noise caused by servo power fluctuations.

This converter ensures stable communication by protecting the sensitive NRF module from voltage instability.

Fig 16. LM2596 buck converter DC-DC

3.1.10. Real-time movement interpolation and gait stability

The second functionality involves ensuring smooth and coordinated leg movement through the use of real-time interpolation. The Ramp[7] library is used to interpolate each joint's movement between calculated positions over time, preventing sudden jumps or jittery motions.

Key components and aspects here:

- Ramp Library[7]: Smooths transitions between joint positions to emulate natural walking.
- PWM Servo Control: Converts joint angles into precise PWM values using Arduino's map() function.
- Tripod Gait Algorithm: Controls the sequence of leg movement for balance and stability.
- Chassis and leg placement: Mechanical adjustments ensure center of mass is balanced.

This interpolation is essential not just for realism, but also to avoid torque spikes that could stall or overload the MG996R servos. It also allowed the robot to walk consistently without tipping over or stalling due to unsynchronized leg motion.

3.2. Design of Hardware solution

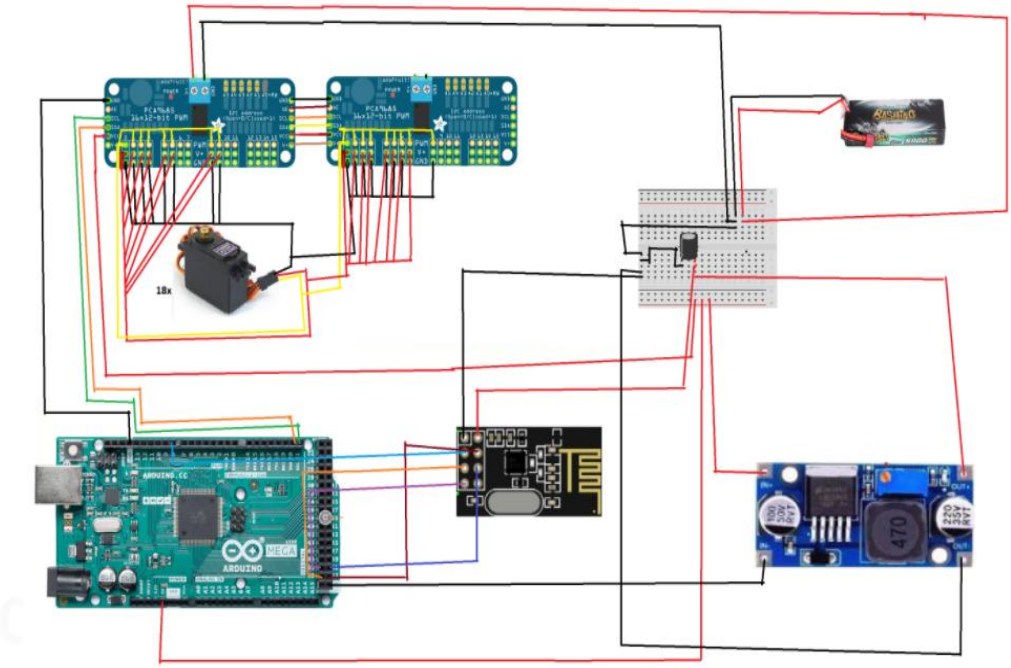

Figure 17. Hardware architecture of the hexapod

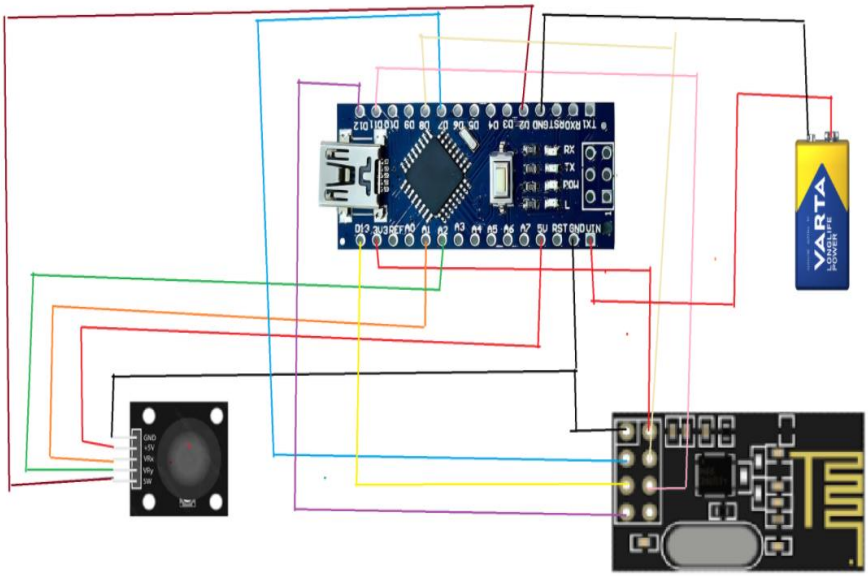

Figure 18. Hardware architecture of the joystick

The flow of information of the hexapod starts with the joystick module , which reads analog input and sends X,Y values so that the arduino Mega can use those values to call a function if X,Y is at certain values , by testing we can see that X and Y becomes certain values and we can tell if the joystick is pushed forward or left based on those values.

The arduino mega recieves and calls a stored function based on X,Y values recieved through radio.

Afterwards it calculates the Inverse Kinematic and sends the appropiate PWM signals to PCA9685 servo drivers using I^2C which then produces a stable PWM signal on its outputs.This architecture reduces processing time on the microcontroller , increasing time accuracy and allows scalable expansion being an essential part for coordinating smooth and synchronized leg movement in the hexapod.

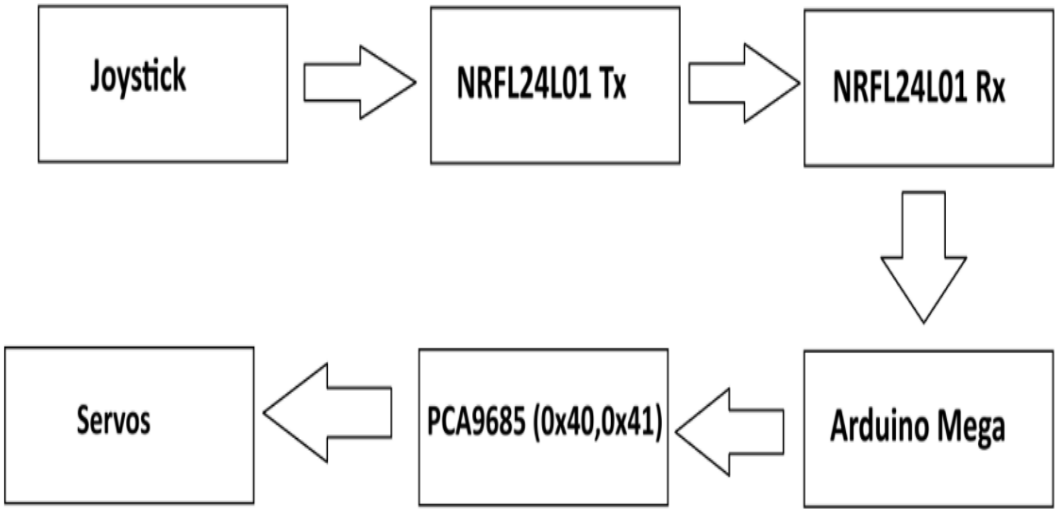

Figure 19. Flow of information block by block for hexapod

3.3. Analysis of software technologies and methods

3.3.1. Motor Control:

Servo motors are the primary actuators responsibles for generating leg movement in the hexapod. Each leg consists of three joints , coxa , femur and tibia , each actuated by a MG996R servo motor and in total , the robot uses 18 servo motors.

Servo motors interpret Pulse Width Modulation signals to determine their rotational position. The MG996R servos expect a signal frequency of 50Hz and pulse width between 500 and 2500 µs, corresponding to angles between 0° and 180°.

Generating such a large number of precise signals is challenging for the arduino Mega due to the fact that it has a limited number of PWM capable pins and lacks the ability to handle synchronized multi-channel PWM generation in real time.

In order to solve this problem , two PCA9685 PWM driver boards were used , each capable of controlling up to 16 servos using timed PWM signals.

These modules receive commands via I^2C bus, and internally generate accurate 12-bit resolution PWM.

Looking at the software part , the joint angles are computed via Inverse Kinematic and are passed to the UpdatePosition function which sets up a smooth interpolation with the help of the Ramp library. When these interpolated joins values are retrieved , they must be translated into PWM compatible values understood by the PCA9685 drivers module.

This conversion is made with the help of the map() function which is as follows:

```
pwm.setPWM(channel, 0, map(angle, 0, 180, SERVOMIN, SERVOMAX));
```

Where SERVOMIN = 150; and SERVOMAX = 600; , these values corresponding to the minimum and maximum pulse widths that the MG996R servos can interpret as 0 ° and 180 ° respectively.

Through this approach, the system integrates inverse kinematics, interpolation, and PWM signal generation to create coordinated, smooth, and biologically inspired leg movements all while operating under the constraints of student-budget hardware.

Although the robot's movement is primarily based on inverse kinematics, it is important to briefly address the concept of direct kinematic, which was also used in the initial development phase during single-leg testing.

3.3.2. Direct Kinematic:

Direct Kinematics refers to the process of determining the position of the robot's end effector based on known joint values and link lengths. Therefore in other words , given the angles of coxa , femur and tibia joints we can compute where the leg's tips is located in the 3D space (X, Y , Z).

Although the final implementation of the project uses only inverse kinematic for motion , direct kinematic was used in the earlier stages of the development phase for testing and validating.

More specifically, when testing a single leg, direct kinematics helped verify that the structure moved as expected.

The direct kinematic can also be calculated with:

- $X=L1*\cos(J1)+L2*\cos(J1+J2)+L3*\cos(J1+J2+J3)$
- $Y=L1*\sin(J1)+L2*\sin(J1+J2)+L3*\sin(J1+J2+J3)$

3.3.3. Inverse kinematic:

Once the inverse kinematic algorithm was implemented and integrated with the gait and controller logic, forward kinematic was eliminated since it served it purpose of testing.

The inverse kinematic refers to the mathematical process of determining the joint angles required to position the robot's end-effector (in this case, the foot of each leg) at a specific point in 3D space.

In the context of this hexapod robot, inverse kinematics is used to calculate the angles for the coxa, femur, and tibia joints so that each leg can reach a desired (X, Y, Z) coordinate with precision.

In the following section, we present a trigonometric breakdown of how inverse kinematics is applied to a 3-joint leg, forming the foundation for coordinated gait patterns such as the tripod gait.

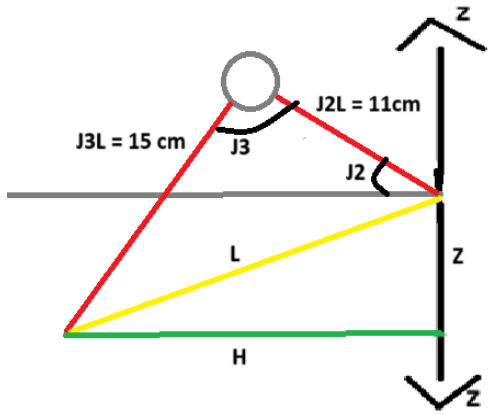

Figure 20. Calculating the inverse kinematic

We need to calculate L , and to do that we can calculate it using Pythagorean theorem which tells us $L^2 = H^2 + Z^2$.

Next we will calculate and solve for angle J3 using the law of cosines $\cos a = (\frac{l1^2+l2^2-r^2}{2*l1*l2})$, which tells us to find angle $J3 = \arccos(\frac{J2L^2+J3L^2-L^2}{2*J2L*J3L})$.

To find angle J2 , we need to look at figure 3 and to find 2 more angles , which are angle A and B, and to find them we can use the law of cosine again.

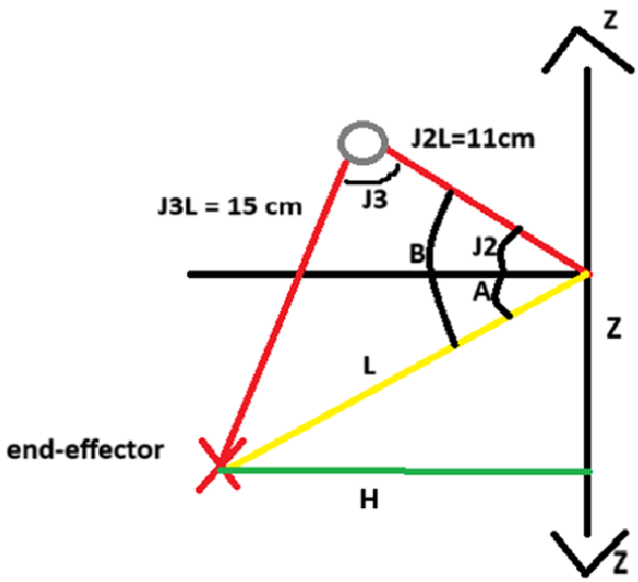

Figure 21. Calculating the inverse kinematic.

To find angle B you need to calculate $B = \arccos\left(\frac{(L^2+J2L^2-J3L^2)}{2*L*J2L}\right)$.

To find angle A you have to calculate $A = \arctan\left(\frac{Z}{H}\right)$.

With these calculations in mind we can find the angle J2 , which is $J2 = B + A$.

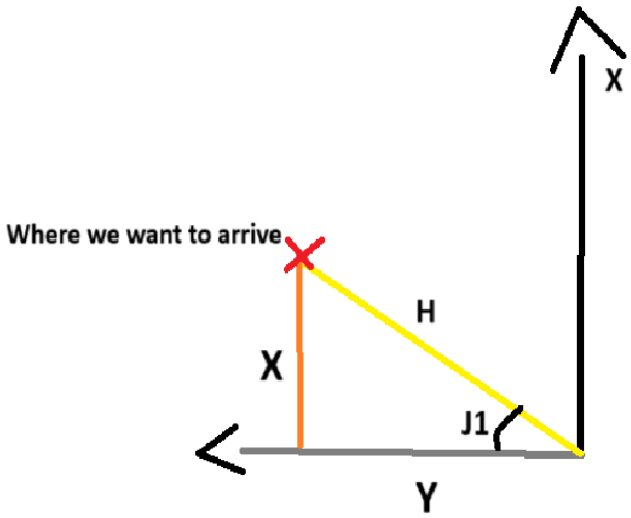

Figure 22. Calculating Inverse kinematic

We assume we want to arrive at a point and in order to do that we need calculate the J1 movements which you do with trigonometry:

- J1 = $(\arctan\left(\frac{X}{Y}\right)$
- H = $\sqrt{X^2 + Y^2}$

With the inverse kinematic calculations in place and joint angles correctly translated into servo commands, the next step involves coordinating the motion of all legs to produce a stable and effective walking pattern.

3.3.4. Movement Control:

This is handled by the movement control system, which in our case is based on the tripod gait strategy.

Tripod gait is one of the most widely used walking algorithms for hexapod robots due to its simplicity, balance, and efficiency. It divides the six legs into two groups of three: while one group is in the swing phase (moving forward) , the other group remains in the stance phase (on the ground, supporting the weight of the robot).

This alternating pattern ensures that the robot always has a stable base of support, minimizing the risk of tipping over.

In this project , legs 1, 3 and 5 form the first tripod group and legs 2,4,6 for the second. During each walking step, the active tripod lifts from the ground and moves its leg according to the stepPath predefined path calculated with inverse kinematics. Once the swing phase is complete , the other group lifts it's leg and the the legs that finished the swing phase pushes the robot forward where in the time where it pushes the leg

group that just lifted its ground , resumes the predefined stepPath from that position until it reaches the end of the stepPath.

In this way it makes the robot move insect like , emulating a live insect moving on the ground.

While the tripod gait ensures coordianted motion and balance, it is the use of interpolation tehnique that makes the movements appear fluid and smooth rather than robotic and jerkish.

Interpolation in this project is implemented at the software level with the help of the Ramp library [7] which allows for gradual transitions between joint angles over time. Rather than commanding each servo to instantly jump from one angle to the next , interpolation creates a smooth progression of the servos movement. This in term reduces mechanical stress on the servo gearboxes , improves realism and stability in motion.

For each leg, the target joint angles calculated by inverse kinematics are passed to interpolated variables called rampDouble which slowly approaches the final angle using a time-based progression. The ".update"[7] function of the Ramp library[7] is called every loop cycle , and the current interpolated angle is sent to the servo using the "map()"[7] function and "pwm.setPwm()"[10].

The interpolation mechanism is crucial when moving multiple legs at the same time during the hexapod gait execution , as it prevents sudden jerks that could seriously destiabilize the robot or even damage the servos.

3.4. Design of software solutions

3.4.1. System architecture

The overall system architecture of the hexapod robot consists of interconnected hardware and software layers, where data flows from the user (via a joystick), through wireless transmission, and finally to the main processing unit for execution. A detailed representation of this architecture, including modules like the Arduino Mega, NRF24L01, PCA9685 drivers, power supply, and sensors, is provided in the figures in the hardware section.

The system architecture of the hexapod robot combines hardware and software components that communicate in a coordinated manner to control motion. At the highest level, the architecture begins with the joystick controller, which acts as the user input device.

The joystick reads analog signals for the X and Y axes and sends this data wirelessly via an NRF24L01 trans-receiver module, connected to an Arduino Nano acting as a transmitter.

On the robot side , the arduino Mega is the central processing unit that continously listens for incoming data with the help of another NRF24L01 module that acts as a reciever.

Upon receiving the joystick input , the arduino mega interprets the values read and calls a function accordingly to X and Y values , which for example if by movign forward the joystick the values would be X>100 and Y < 100 , then it would call the moving forward command.

When calling any movement command the arduino Mega does the following:

- Executes the inverse kinematic for each leg to determine the required joint angles based on predefined trajectories.

- It applies interpolation using Ramp library to ensure smooth transitions between positions, minimizing mechanical stress on the servos.
- Sends the result angles as PWM signals to the appropriate channels of the PCA9685 servo driver boards via the I^2C protocol.

Two PCA9685 boards are used, each capable of controlling up to 16 channels, allowing to control 18 servos which are divided between 2 PCA9685 , so that while executing the gait , one PCA9685(addressed as 0x40) is responsible for a groupA and another driver (.adrresed as 0x41)module is responsible for groupB.

The servo motors (MG996R) execute the physical movements by rotating the joints to the specified angles, resulting in coordinated leg motion based on a tripod gait. Three legs are always in motion while the other three provide stability, simulating insect-like locomotion.

Power distribution is managed by a Li-Po 7.4V 5000mAh battery, with a buck converter (LM2596) used to provide a steady 3.3V to the NRF module, stabilized with a 47µF capacitor to prevent voltage dips.

A complete schematic of the system connections and data flow is already provided in the hardware architecture section.

This illustrates how signals and power are routed across modules, ensuring proper timing, isolation, and performance.

In system architecture , software implementation and architecture is as important as hardware implementation and depending on the project , even more important.

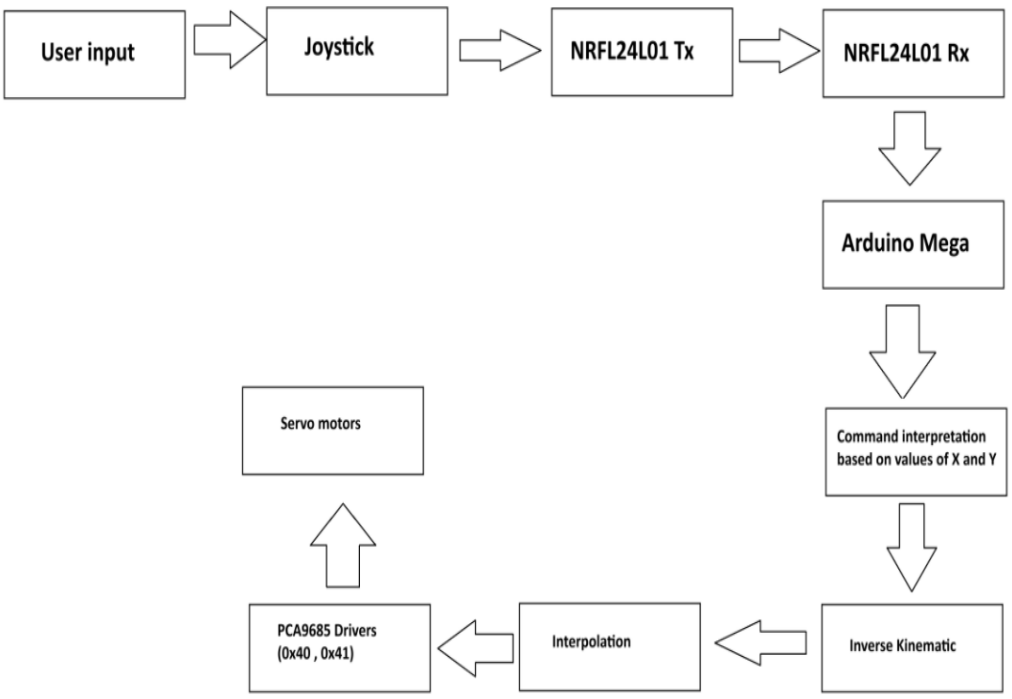

Fig 23. System architecture

3.4.2. Low-level software architecture:

The low-level software architecture defines how the robot interacts directly with hardware components such as servos, drivers, and sensors. This layer is primarily concerned with setting up and managing the essential operations of hardware modules and ensuring accurate signal transmission and control.

In this project, the microcontroller initializes and configures two PWM driver boards (PCA9685) over the I²C communication protocol. These boards are responsible for generating the correct PWM signals required to position each servo motor accurately. Each board is assigned a unique I²C address to allow simultaneous control of multiple channels, making it possible to operate all 18 servos individually.

To ensure smooth motion, servo signals are calculated as angle values and then converted to PWM pulse lengths. This translation ensures that each joint reaches its intended position within a safe and expected range of motion. Timing and signal consistency are especially important to prevent erratic movements or mechanical strain on the robot's joints.

In addition to the servo control, the system includes a motion smoothing mechanism using an interpolation strategy. This method ensures that changes in servo positions are gradual, mimicking fluid and natural leg motion. Instead of instantly snapping a servo to a new angle, the signal transitions progressively, which improves the stability of the robot and reduces mechanical stress.

This low-level architecture is fundamental for ensuring the robot can reliably interpret high-level motion instructions and convert them into accurate joint commands.

3.4.3. High Level software architecture:

The high-level software architecture focuses on how the robot interprets and processes user commands, coordinates gait logic, and manages communication across system components. It defines the logical structure and behavior that governs the robot's overall functionality.

At the core of the system is the command interface between the user and the robot, facilitated by a wireless joystick. The joystick, through an Arduino Nano and an NRF24L01 transceiver module, sends analog signals (representing directional input) to the robot. Onboard, an Arduino Mega receives these commands using a corresponding NRF24L01 receiver module.

Once received, the robot interprets the input values to determine the intended action—such as moving forward or standing still. These high-level commands are translated into positional targets for the robot's legs, initiating a movement cycle.

The central component of this movement logic is the tripod gait. This strategy divides the six legs into two alternating groups, ensuring that three legs are always on the ground to maintain stability while the other three move. The logic for this gait is handled by the microcontroller and is executed based on the interpretation of joystick inputs.

Following the determination of movement, inverse kinematics is used to calculate the precise joint angles required to move each leg to its target position. These values are then passed down to the low-level layer for actuation.

In parallel, the system also maintains timing synchronization between leg movements using delay management and state control. This ensures that transitions between movement phases (swing and stance) occur smoothly and without overlap, preserving balance and rhythm.

This layer of the software handles coordination, user interaction, and logical flow, enabling the robot to respond in real-time to remote commands while preserving movement efficiency and mechanical safety.

Chapter 4 : Implementation and testing

4.1. Hardware implementation of the prototype

The hardware implementation process began with assembling the mechanical structure of the hexapod robot.

I first ordered a hexapod chassis, which consisted of an upper and lower plate and joint connectors for six legs. The two plates were connected using screws, providing a stable frame for the robot.

Each joint chasis was fixed moderatly secured to the adjacent joint chasis , and MG996R was to be put inside of a joint chassis and fixed with a servo head to the main body as well as put on the bottom of the join chasis a bearing so that it can use smooth mechanical movements.

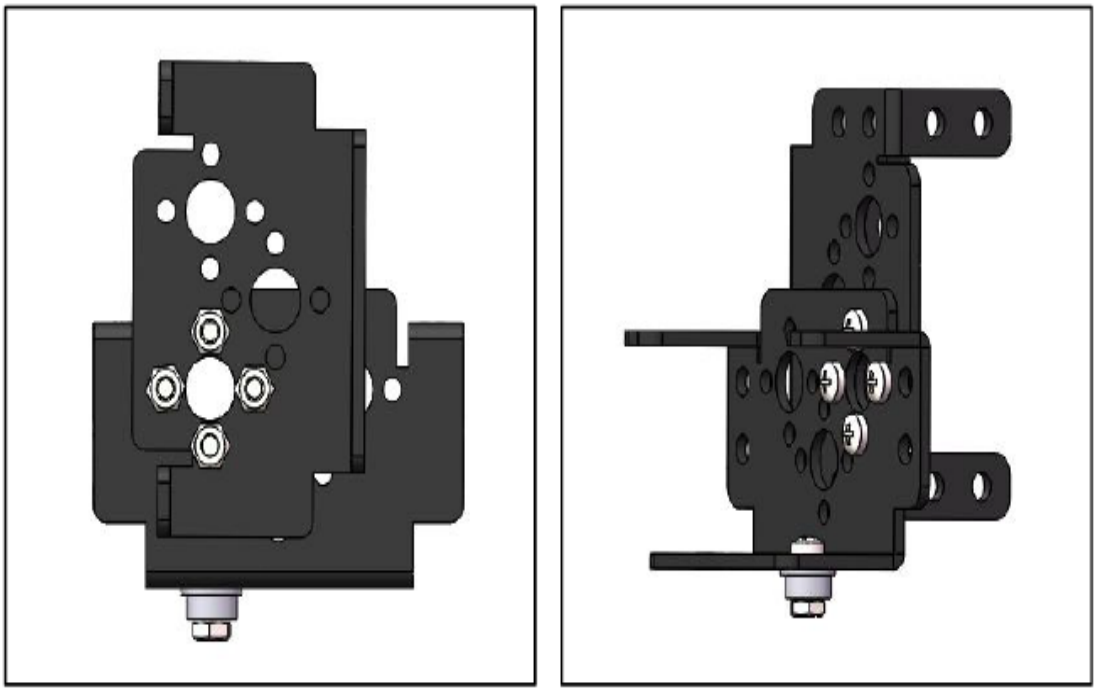

Fig 24.Joint connexions [18]

I initialized each servos to 90 degrees using a basic arduino setup without any drivers just an arduino UNO. This made sure that all the servos started from a neutral angle , which is critical in symetrical and balanced leg movement.

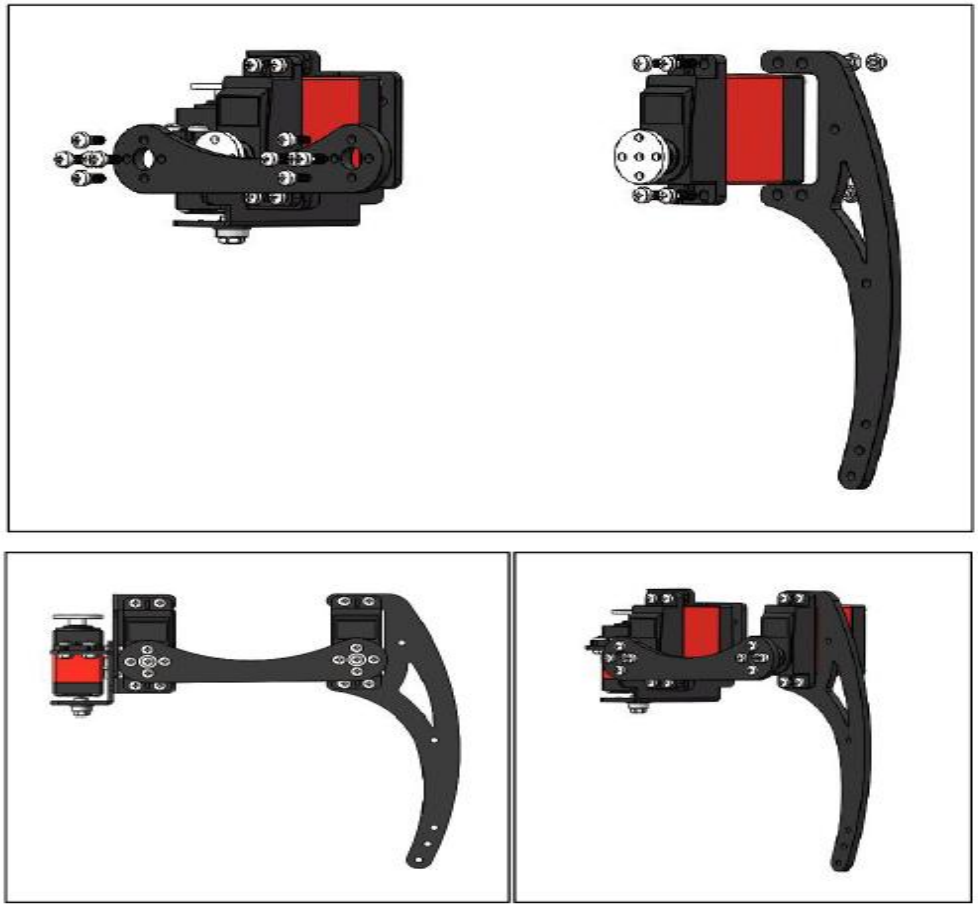

Fig 25.Full leg connexion [18]

Once the mechanical assembly for the legs were complete , i mounted the Arduino Mega 2560 on the upper chassis plate after connecting the 4 pillars which held the upper chasis connected to the lower chasis plate of the body. The Mega was then connected to two PCA9685 servo drivers via the I²C interface to allow control of all 18 servos (3 per leg). To allow each driver to be addressed individually, I soldered the address-select jumper (A0) on the second PCA9685, assigning it address 0x41 while the first remained at 0x40.

For remote control, an NRF24L01 wireless module was added to the robot.

This allowed for wireless joystick input using a transmitter system built with an Arduino Nano, joystick module, NRF24L01 transmitter, and a 9V battery. The joystick sends X and Y values, which are interpreted by the Arduino Mega to generate walking commands.

Powering the robot required some experimentation. Initially, I attempted to power the NRF module using a 7.4V Li-Po battery with a buck converter, but this resulted in communication failures due to voltage instability caused by the high current draw from the servos. To solve this, I supplied the NRF module from the 5V pin of the Arduino Mega, stabilized by a 47µF capacitor, which successfully resolved the issue.

Finally, the servo driver and communication system were tested. After verifying servo movement through test scripts, I connected the entire system and performed successful remote walking trials with real-time joystick control after extensive modifications to the main code.

4.2. Implementation and testing of the software system

The practical part of the low-level software architecture is the individual modules, functions, and libraries directly interacting with the hardware components.

At this level, the software is closely tied to hardware, focusing on initialization, signal generation, servo control, and precise motion control mechanisms.

Initially, the servo drivers are set up using the Adafruit_PWMServoDriver library [10].

This library simplifies controlling multiple servos by generating precise PWM signals via the I²C protocol. The robot uses two PCA9685 boards, each configured with a distinct I²C address, allowing simultaneous control of up to 32 servos:

```
// ptr prima placa
Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver(0x40);
// ptr a doua placa
Adafruit_PWMServoDriver pwm2 = Adafruit_PWMServoDriver(0x41);
```

Fig 26. Initialising drivers

Following driver initialization, each board is set to operate at 50 Hz, matching the recommended PWM frequency for MG996R servomotors:

```
pwm.begin();
pwm.setPWMFreq(50);
pwm2.begin();
pwm2.setPWMFreq(50);
```

Fig 27.Initialising drivers according to drivers requirments

To control each servo, PWM values are computed using Arduino's built-in map() function.

This function converts a desired angle (0° to 180°) into a pulse length between SERVOMIN and SERVOMAX, which represent the servo's minimum and maximum allowablePWMvalues.

```
pwm.setPWM(channel, 0, map(angle, 0, 180, SERVOMIN, SERVOMAX));
```

Fig 28.Calculating PWM values

Lastly, to ensure smooth servo movement and avoid sudden mechanical strain, the software implements interpolation using the Ramp library [7]. Each joint angle is interpolated over time, providing a gradual transition between current and target positions:

```
// librarie ramp pentru interpolare
rampDouble J1Tar[6] = 0.0;
rampDouble J2Tar[6] = 0.0;
rampDouble J3Tar[6] = 0.0;
```

Fig 29. Defining legs to be interpolated

Regarding command interpretation , the joystick sends the values of X,Y through the arduino nano which passes onto the NRF24L01 that acts as a transmitter and the NRF module that is on the robot acts as a reciever listening on a radio frequency you set.

```
radio.begin();
radio.setPALevel(RF24_PA_LOW);
radio.setChannel(108);
radio.openWritingPipe(address);
radio.stopListening();
```

Fig 30.Transmitter

On the moment the values are recieved , depending on the values, the micro controller arduino Mega will decide to move the robot in a direction , following the classic and reliable tripod gait.

```
radio.setPALevel(RF24_PA_LOW)
radio.setChannel(108);
radio.startListening();
```

Fig 31 .Setting the robot to listen to commands from the transmitter

Tripod gait is the ideea of moving 3 legs and 3 legs are to remain stationary to keep the balance of the robot , and this movement from the tripod gait will be executed thanks to the inverse kinematic function that will calculate based on the function called the inverse kinematic so that the robot can move in a direction.

```
const int groupA[] = {0,2,4};
const int groupsizeA = sizeof(groupA) / sizeof(groupA[0]);
const int groupB[] = {1,3,5};
const int groupsizeB = sizeof(groupB) / sizeof(groupB[0]);
```

Fig 32.Defining the groups of the tripod Gait

This movement is further upgraded by the interpolation from the Ramp library that has to .update constantly while executing , emulating a vivid and insect like walking. While the NRF gets supplied of current it will continue to listen to radio on a specifiec frequency that both modules are atunned so that they can interact with each other.

```
J1Tar[leg].update();
J2Tar[leg].update();
J3Tar[leg].update();

double j1Val = J1Tar[leg].getValue();
double j2Val = J2Tar[leg].getValue();
double j3Val = J3Tar[leg].getValue();

// Only leg 0, 2, 4
if (leg == 0 || leg == 2 || leg == 4) {
    pwm2.setPWM(channel[leg][0], 0, map(90 - j1Val, 0, 180, SERVOMIN, SERVOMAX));
    pwm2.setPWM(channel[leg][1], 0, map(90 - j2Val, 0, 180, SERVOMIN, SERVOMAX));
    pwm2.setPWM(channel[leg][2], 0, map(j3Val + J3_LegAngle - 45, 0, 180, SERVOMIN, SERVOMAX));
}
```

Fig 33.Interpolating each leg in a group

Movement timing is controlled using millis()-based delays to synchronize steps and push phases. This avoids blocking delays and ensures that gait updates and servo commands occur in real time without interfering with communication or other tasks.

```
if (groupFinished && millis() - lastStepTime > stepDelay) {
  lastStepTime = millis();
  stepindex++;

  // wrap & reset mid-swing flags
  if (stepindex >= pathlength) {
    stepindex = 0;
    counter_movement++;
    hasLifted = hasPushed = false;
  }
```

Fig 34.Timing control

The system maintains internal states for each leg using the LegStatus structure, which tracks step index, movement status, and remaining path. This helps coordinate leg motion, avoid conflicts, and smoothly alternate between tripod groups.

```
struct LegStatus {
  double (*path)[3];    // Pointer
  int pathLength;
  int stepIndex;
  unsigned long lastStepTime;
  bool isMoving;
  int stepsRemaining;
  int lastStepIndex = -1;


};

LegStatus legs[6];
```

Fig 35.Control flow

4.2.1. Implementation and testing of motor control:

The motor control system in the hexapod robot was implemented using two PCA9685 servo driver modules, each capable of controlling up to 16 PWM channels.

These drivers were necessary due to the limited number of PWM pins available on the Arduino Mega, which could not independently drive all 18 MG996R servos. Each PCA9685 board was assigned a unique I²C address 0x40 for the first board and 0x41 for the second by soldering the address configuration pin on the first board. The drivers were initialized in the setup phase using the Adafruit PWM Servo Driver library:

```
Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver(0x40);
Adafruit_PWMServoDriver pwm2 = Adafruit_PWMServoDriver(0x41);

pwm.begin();
pwm.setPWMFreq(50);
pwm2.begin();
pwm2.setPWMFreq(50);
```

Fig 36.Setting up the drivers

A frequency of 50 Hz was selected to match the MG996R servo's requirements. Each servo is assigned to a specific PWM channel, and angles are translated into PWM values using Arduino's built-in map() function. The following function call sets the PWM signal:

```
pwm.setPWM(channel, 0, map(angle, 0, 180, SERVOMIN, SERVOMAX));
```

Fig 37.Setting pwm signal

Here, SERVOMIN and SERVOMAX correspond to the pulse lengths that define 0° and 180°, calibrated as 150 and 600 respectively for the MG996R.

During testing, all servos were first centered at 90° to ensure symmetrical installation of servo horns.

This standardization was critical to avoid mechanical offsets when the robot starts moving.

The motor control subsystem was then tested and validated by sending direct PWM commands to the servo motors through the PCA9685 driver boards.

Each servo was positioned manually using fixed angles to verify joint response and orientation.

During the setup phase, all servos were initialized to specific reference positions to ensure mechanical symmetry and alignment:

- Femur servos were set to ~140° to lift the leg.

- Tibia servos were set to ~90° to form a stable stance.

This positioning was done using pwm.setPWM() and pwm2.setPWM() for the two boards, mapped through channel[i][j] for each leg and joint.

Servo movement was confirmed visually, ensuring each motor responded correctly to its command.

```
delay(100);
pwm.setPWM(0,0,map(90, 0, 180, SERVOMIN, SERVOMAX));
delay(100);
pwm.setPWM(1,0,map(140, 0, 180, SERVOMIN, SERVOMAX));
delay(500);
pwm.setPWM(2, 0, map(90, 0, 180, SERVOMIN, SERVOMAX));
delay(100);
pwm.setPWM(3, 0, map(90, 0, 180, SERVOMIN, SERVOMAX));
delay(100);
pwm.setPWM(4, 0, map(40, 0, 180, SERVOMIN, SERVOMAX));
delay(500);
pwm.setPWM(5, 0, map(90, 0, 180, SERVOMIN, SERVOMAX));
delay(100);
pwm.setPWM(6, 0, map(90, 0, 180, SERVOMIN, SERVOMAX));
delay(300);
pwm.setPWM(7, 0, map(40, 0, 180, SERVOMIN, SERVOMAX));
delay(500);
pwm.setPWM(8, 0, map(90, 0, 180, SERVOMIN, SERVOMAX));
Serial.println(" finalizare initializare prima grupa pwm");
delay(2000);
```

Fig 38. GroupA initialization

```
pwm2.setPWM(0,0,map(90, 0, 180, SERVOMIN, SERVOMAX));
delay(100);
pwm2.setPWM(1,0,map(140, 0, 180, SERVOMIN, SERVOMAX));
delay(500);
pwm2.setPWM(2, 0, map(90, 0, 180, SERVOMIN, SERVOMAX));
delay(100);
pwm2.setPWM(3, 0, map(90, 0, 180, SERVOMIN, SERVOMAX));
delay(100);
pwm2.setPWM(4, 0, map(140, 0, 180, SERVOMIN, SERVOMAX));
delay(500);
pwm2.setPWM(5, 0, map(90, 0, 180, SERVOMIN, SERVOMAX));
delay(100);
pwm2.setPWM(6, 0, map(90, 0, 180, SERVOMIN, SERVOMAX));
delay(100);
pwm2.setPWM(7, 0, map(90, 0, 180, SERVOMIN, SERVOMAX));
delay(500);
pwm2.setPWM(8, 0, map(90, 0, 180, SERVOMIN, SERVOMAX));
delay(100);
pwm2.setPWM(9, 0, map(40, 0, 180, SERVOMIN, SERVOMAX));
delay(100);
pwm2.setPWM(10, 0, map(90, 0, 180, SERVOMIN, SERVOMAX));
delay(500);
pwm2.setPWM(11, 0, map(90, 0, 180, SERVOMIN, SERVOMAX));
Serial.println(" Final initializere ultima grupa  ");
delay(4000);
```

Fig 39. Group B initialisation

The first PCA9685 board (address 0x40) and the second (address 0x41) were tested after individually setting their I²C addresses by soldering the appropriate address pin on the first board.

Although a full-angle sweep was not conducted programmatically, the static angle tests ensured that motor control was functional and reliable before proceeding to kinematic implementation.

4.2.2. Implementation of the Direct kinematic

Direct kinematics was briefly used during the early stages of development to test the motion of a single leg.

By manually setting the joint angles (coxa, femur, tibia), the resulting position of the leg tip in 3D space was observed and compared to expected values. This helped verify the mechanical behavior of the joints.

No code or formulas were retained, as this approach was later replaced entirely by inverse kinematics for actual robot movement.

However, this initial phase was essential for confirming servo orientation, range, and mechanical consistency across legs.

4.2.3. Implementation and Testing of Inverse Kinematics:

The inverse kinematics (IK) algorithm was a core part of the robot's walking behavior. It was implemented to compute the required angles for the coxa, femur, and tibia joints based on a desired (X, Y, Z) position of the foot. This allowed precise and coordinated leg placement.

The implementation used trigonometric relationships, specifically the law of cosines and basic triangle geometry, to convert input coordinates (X, Y, Z) into angles. These angles were then transformed into PWM values and sent to the corresponding servos using the Adafruit PCA9685 driver boards:

To validate the correctness of the algorithm, initial testing was conducted on a single leg using predefined coordinates. The output joint angles were printed to the Serial Monitor and visually compared with the leg's actual position. This allowed for debugging and fine-tuning of constants such as segment lengths and angle offsets.

Once validated, the function was integrated into the main gait engine, where the legs followed a step path defined by the stepPath[] array. During runtime, the angles were smoothed using interpolation from the Ramp library, ensuring a natural and insect-like movement.

The results confirmed that the inverse kinematics worked as intended:

- Legs moved smoothly along the predefined trajectory.
- The robot could consistently lift and place its feet with minimal deviation.
- Transitions between support and swing phases of the gait were stable.

This module served as the foundation for all leg motion, and its successful implementation was crucial for the tripod gait and directional control of the robot.

```
double J1 = atan(X / Y) * (180 / PI);
double H = sqrt((Y * Y) + (X * X));
double L = sqrt((H * H) + (Z * Z));
double J3 = acos(   ((J2L * J2L) + (J3L * J3L) - (L * L))   /   (2 * J2L * J3L)   ) * (180 / PI); //  cos alpha
double B = acos(   ((L * L) + (J2L * J2L) - (J3L * J3L))   /   (2 * L * J2L)   ) * (180 / PI);
double A = atan(Z / H) * (180 / PI);
double J2 = (B + A);
```

Fig 40.Implementing Inverse Kinematic

It was multiplied with $\left(\frac{180}{\pi}\right)$ to convert from angles to radians.

4.3. Hexapod system testing

After assembling and validating each component individually, a full system integration test was conducted to evaluate the robot's overall functionality, stability, and responsiveness.

The hexapod was placed on a carpet due to lack of grip on laminated surfaces and tried to do a setup trying to push itself from the ground but it concluded that the robot cannot get up on its feet by itself . Afterwards i let him setup and put him myself on the feet , when i concluded that it can't maintain it's balance due to insufficient torque to get up it's own. I proceeded with the first attempt of the setup of the legs , which was a failure as the robot failed to maintain it's balanced.

Fig 41. Hexapod before connections

The second testing phase i closed the distance between legs mechanically, making so that the weight would be centered close to the legs , this time the robot succesfully maintained it's balanced however it wasn't able to move as after the first step it sliped , mostly because the exectuion time had a delay at that time and it took quite a bit of time to move and probably maintained position too long, that might have been one of the reasons.

The third second testing , i've decided to further mechanically move each leg closer to each other and tried testing it again , however this time the supply for the NRF trans-reciever module became unstable due to voltage drops , fact at the time that i didn't know , and i've tried multiple things to remedy it such as putting a buck converter

from the VCC from the 7.4V 5000mAh battery which didn't work because of voltage spikes since the drivers were also using the battery and produce quite a bit of noise. I've also tried to put the 3.3V pin from the arduino Mega but to no avail , this challange still kepts pestering me until i've decided to try to put the 5V from the arduino Mega to pass it through a buck converter so that the output would be 3.3V and the output would pass through a 47µF in order to filter the noise further which this worked.

On the fourth testing , after fixing the reciever module , the robot became promising as i've powered him up and stabilised him on it's legs and it maintained balance , i've also modifed it's delay to be very short, and i've run it and i've kept moving him for 15 second straight until it sliped , which was a great success and the project could be concluded as i've succeded in using an under torque servos for a heavier frame and made it possible to move it and demonstrate itself with the help of the tripod gait which maintained an almost perfect balance while walking.

Fig 42.Hexapod final stage of testing

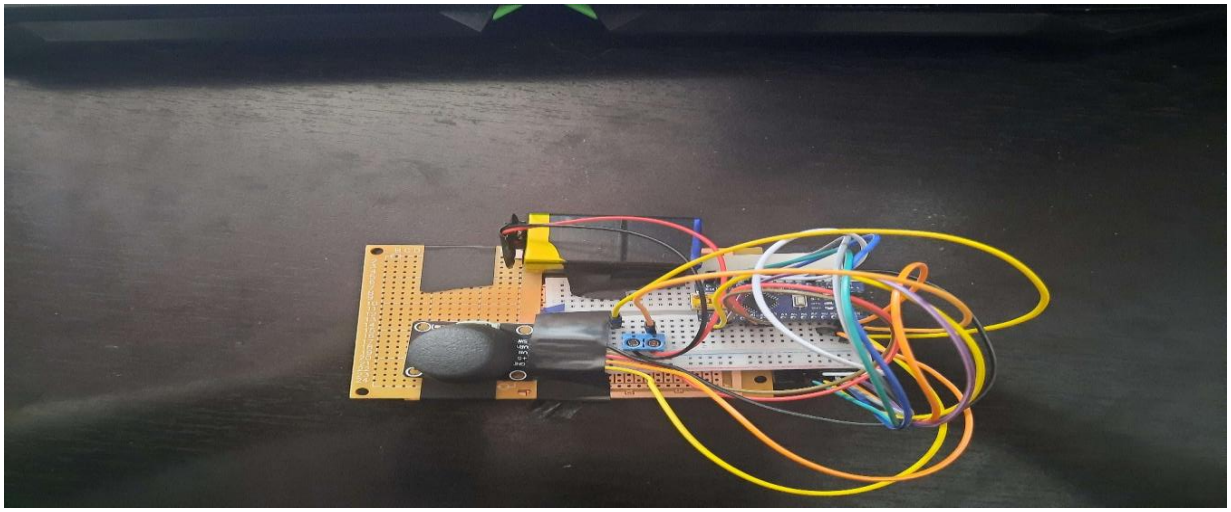

Fig 43.Custom made joystick

Chapter 5 Conclusion

5.1. Achieved Results

The development of the hexapod robot was a mild succes in terms of technical and in temrs of education it was a success. Starting from the ground up, the robot was constructed using affordable and accessible hardware such as MG996R servos, PCA9685 PWM drivers, NRF24L01 modules, and an Arduino Mega microcontroller.

Despite the cost-effective nature of these components and their known limitation particularly the relatively low torque of the servos the final prototype was capable of performing stable walking motions using the tripod gait principle, a key aspect in insect-inspired robotics.

The project not only covered the mechanical assembly and electronic integration but also required software design, algorithm implementation, and iterative testing.

Through the implementation of inverse kinematics and interpolation techniques, the robot was able to translate high-level coordinate targets into smooth and accurate leg movements.

Communication between the joystick and robot was achieved via NRF24L01 transceivers, allowing for remote operation and responsive control.

Even though the system initially struggled with power fluctuations and mechanical stability, these issues were progressively resolved through engineering workarounds, such as optimizing the leg geometry, using interpolation ramps to smooth transitions, and filtering power to the communication modules.

Ultimately, the robot could perform a stable walk cycle across a carpeted surface for extended periods without tipping over demonstrating not just functional success but also real-world applicability.

5.1.1. Degree of Achievement of Proposed Objectives :

The initial objectives of the project were well defined and can be evaluated as follows:

- Design and assemble the mechanical structure of the hexapod:

  The robot was successfully built from a purchased chassis, with careful alignment and calibration of all six legs. Servos were mechanically zeroed and mounted accurately untill eventualy the initial position were changed.

- Implementation motor control through AdaFruit PCA9685 drivers [10]:

The MG996R servo motors were controlled using PWM signals generated by two PCA9685 modules, each communicating via I²C with the Arduino Mega. Initialization and configuration were tested and verified.

- Build and test inverse kinematics for 3DOF legs:

Mathematical models were created and implemented in Arduino code to compute joint angles (coxa, femur, tibia) based on a desired XYZ position. These models were confirmed during single-leg tests and full tripod coordination.

- Create movement logic using the tripod gait :

The gait logic was based on alternating leg groups (1,3,5 and 2,4,6) to maintain balance while executing forward steps. Testing showed stable movement under this gait.

- Implementation of linear interpolation using the Ramp library for smoother motion:

Rather than jumping from one position to another, joint angles were ramped smoothly using the Ramp library, producing more natural, insect-like movement and reducing mechanical strain.

- Enable remote control through NRF24L01 modules and joystick:

The joystick's X and Y values were wirelessly transmitted to the robot using NRF24L01 modules. After solving early power stability issues, the communication was responsive and effective.

5.1.2. List of personal contributions:

- Selected, purchased, and assembled the full mechanical chassis from available kits and components
- Calibrated servos mechanically and in software to ensure consistent zero positions across all legs
- Designed the system architecture (hardware and software) including two PWM drivers, NRF modules, power routing
- Researched and coded inverse kinematics for a 3-joint robotic leg
- Implemented smooth joint interpolation using the Ramp library
- Developed tripod gait logic and integrated it with inverse kinematics and servo control
- Wrote and tested transmitter code for Arduino Nano + joystick input

- Designed and tested receiver code to interpret commands and drive legs accordingly
- Solved NRF power issues through voltage filtering and buck converter configuration
- Iteratively tested mechanical configuration, improved stability by adjusting leg spacing
- Performed and documented full system integration tests

5.2. Future development direction

Future steps for developing further the hexapod would be implementing a PID loop with the help of a gyroscope , further increasing it's stability on uneven terrain.

Another step i would take is implement a pi co microcontroller that implements a camera with potenially face recognition.

Other upgrades that would surely be implemented would be stronger servos in terms of torque , this being a vital in future prototypes that would be designed for explorations.

Perhaps another upgrade for the micro controller would be the upgrade of using a raspberry pi like [19]Rodrigo Barbaosa Letong with his colleagues did in the paper „ Development of an Autonomous Hexapod Robot with Digital Image Processing Implemented on Raspberry Pi" where it used the raspberry pi with digital image processing.

Bibliography

[1] A. Latif, H. A. Widodo, R. Rahim and K. Kunal, "Implementation of Line Follower Robot based Microcontroller ATMega32A," Journal of Robotics and Control (JRC), vol. 1, no. 2, pp. 70-74, 2020.

[2] V. Perricone, C. Santulli, F. Rendina, and C. Langella, “Shape optimization of high-speed rail by biomimetic,” in *ICME’17 MATEC Web of Conferences*, 2017.

[3] Tassos Natsakis "Robot Controlled Systems" book

[4] TowerPro, "MG996R High Torque Servo Motor – Datasheet," TowerPro Robotics,

[5] S.Aydin, L. Gokrem and M.S. Cam , "Hexapod Robot Design and Performance Comparison of Fuzzy and PID Control Methods"

[6] Ahmed E. Salman , Magdy Roman "

[7] Sylvain GARNAVAULT, Ramp library used for interpolation "https://github.com/siteswapjuggler/RAMP"

[8] HandsonTechnology,"https://www.handsontec.com/dataspecs/motor_fan/MG996R.pdf"

[9] SparkfunElectronics,NRF24L01, "https://cdn.sparkfun.com/datasheets/Wireless/Nordic/nRF24L01_Product_Specification_v2_0.pdf"

[10] Adafruit PWM servo driver library, "https://learn.adafruit.com/16-channel-pwm-servo-driver/using-the-adafruit-library"

[11] Hector Montes ,Lisbeth Mena ,Roemi Fernandez,Javier Sarria , "Hexapod Robot For Humanitarian Demining"(research gate website)

[12] Boston Dynamics , Spot Robot " https://bostondynamics.com/products/spot/"

[13] Trossen_Robotics_,PhantomX_Hexapod_, https://www.researchgate.net/figure/PhantomX-AX-Metal-Hexapod-Mark-II-7_fig1_357201772"

[14] 3D printed hexapod "https://free3d.com/3d-model/hexapod-dsd-2-684498.html"

[15] Saranli, U., Buehler, M., & Koditschek, D.E. (2001). *RHex: A Simple and Highly Mobile Hexapod Robot*. IJRR.

[16] Albiez, J., Berns, K. (2004). Control System of the Walking Robot LAURON IV. CLAWAR.

[17] Hutter, M. et al. (2016). *ANYmal – A Highly Mobile and Autonomous Quadrupedal Robot*. IROS.

[18] YFROBOT Electronics Workshop

[19] Letang, Rodrigo & Mori, Ricardo & Campo, Alexandre. (2015). Development of an Autonomous Hexapod Robot with Digital Image Processing Implemented on Raspberry Pi. 10.20906/CPS/COB-2015-0511.

[20] 3D printed hexapod tutorial " https://www.instructables.com/3D-Printed-Hexapod/ "